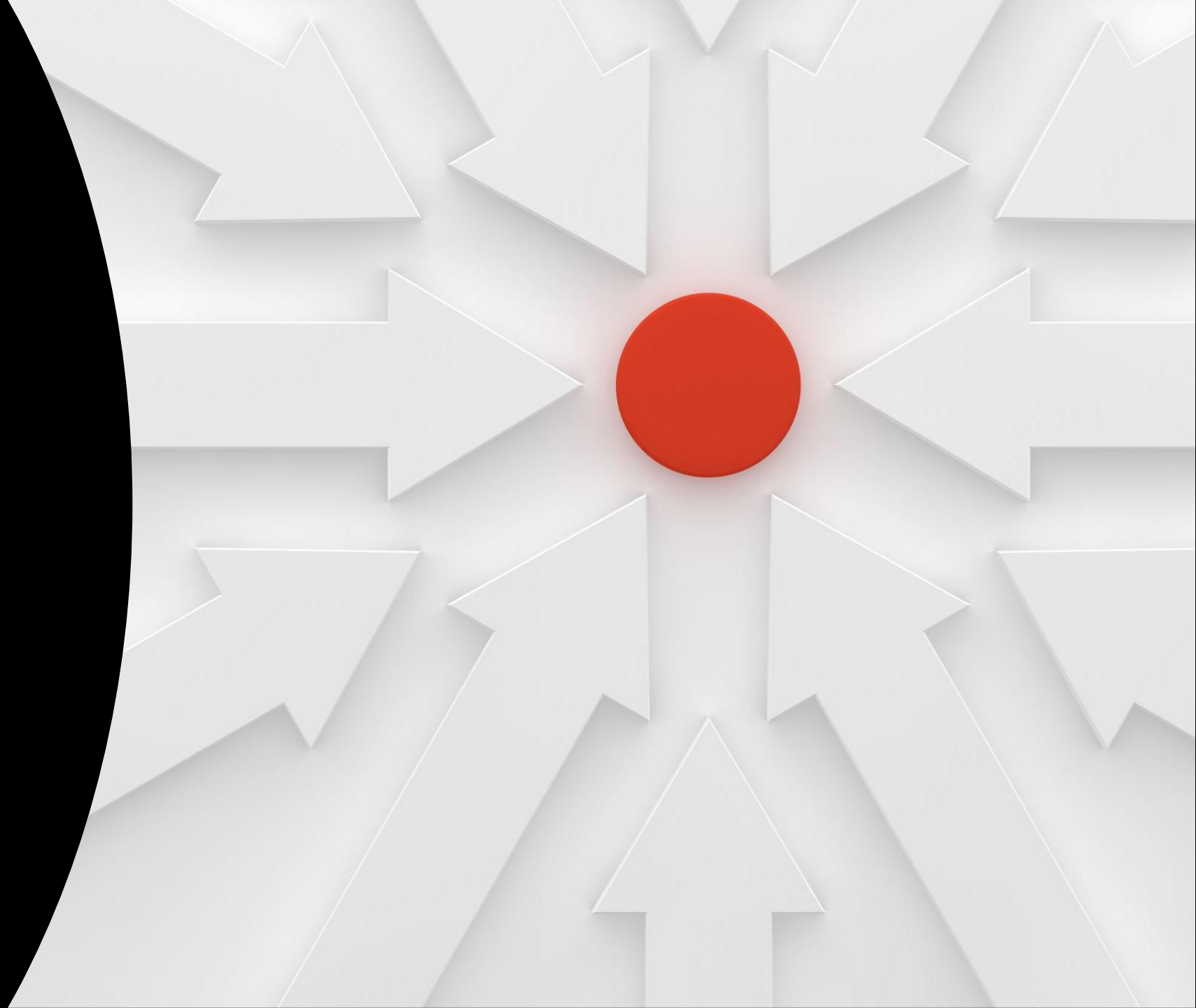


# Pointers

ITP 2024

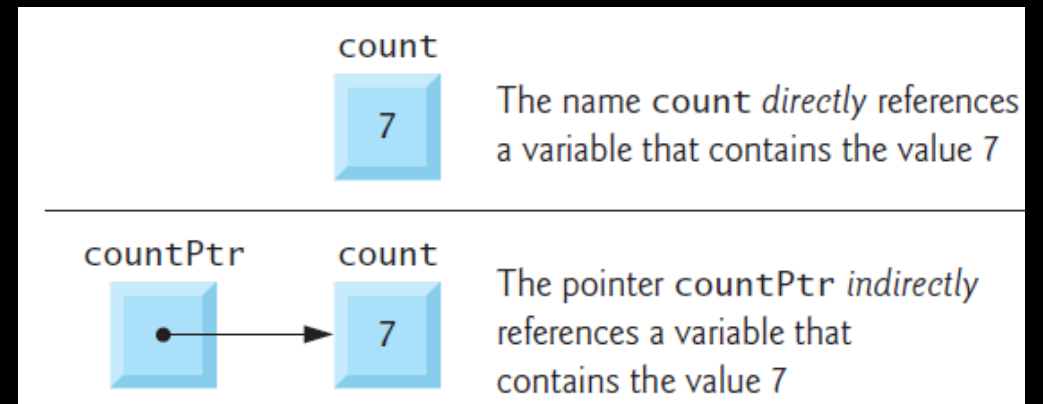


# Pointers

- What?
- Why?
- How?

# What are Pointers?

- Pointers are variables that contain the address of another variable
  - Normal variables contain a specific value (direct reference)
- The pointer points to that variable
- Referencing a value through a pointer is called indirection (indirect reference).
  - A variable name directly references a value, and a pointer indirectly references a value



# Why Pointers?

---

**Memory efficiency:** Pointers can handle large data efficiently. Passing large structures or arrays by pointer instead of value prevents the overhead of copying entire blocks of memory (allows pass-by-reference).

---

**Memory management:** Pointers are crucial for dynamically allocating memory at runtime. Useful for creating dynamic data structures.

---

**Manipulating Arrays and Strings:** Pointers can be used to access array elements directly. Pointer arithmetic allows you to traverse arrays efficiently.

---

**Function Pointers:** Pointers can be used to point to functions, allowing functions to be passed as arguments, stored in arrays, or dynamically assigned at runtime.

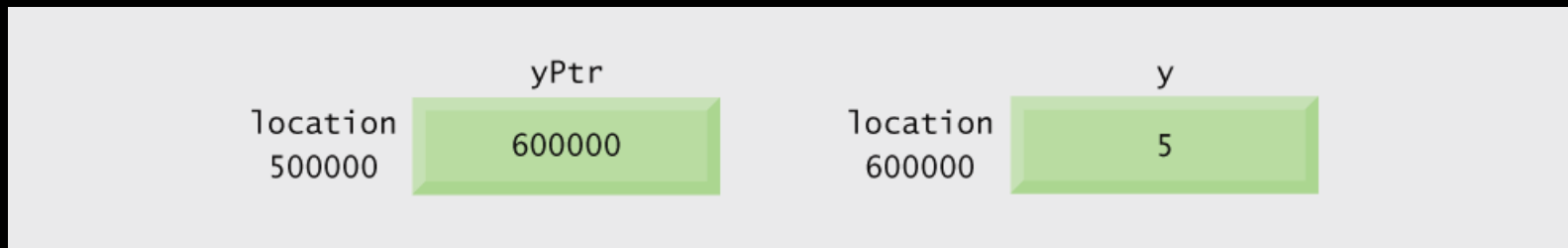
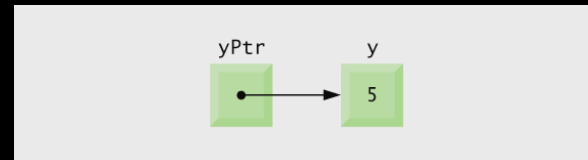
---

# How to use Pointers?

- Needs to be declared before use. Read as: *countPtr is a pointer to an integer.*
  - `Int *countPtr;`
- The `*` is associated with the variable. The `*` doesn't distribute to each var.
  - `int *a, b;` // a is a pointer variable, b is a regular variable
  - `Int *c, *d;` // Both c and d are pointers
- Pointers can be initialized to NULL, 0, or any memory address
- A pointer with NULL value does not point to anything
- NULL is a symbolic constant with value 0 and is defined in `<stddef.h>`
- Initializing a pointer to 0 is equivalent to initializing it to NULL.
- The constant NULL is preferred because it emphasizes that you're initializing a pointer rather than a variable that stores a number

# Pointer Operators: Address(&)

- Address operator (&) gives address of variables
  - `int y = 5;`
  - `int *yPtr = &y;`
  - `yPtr` “points to” `y`



# Good Practices when using Pointers

- .Include the letters **ptr** in pointer variable names to make it clear that these variables are pointers and thus need to be handled appropriately.
- Initialize pointers to prevent unexpected results.

# Pointer Operators: Indirection(\*)

- Indirection operator or dereferencing operator (\*) gives the content of the memory location (value of the variable to which pointer points to)
  - `int x = 10;`
  - `int *xPtr = &x;`
  - `int y = *xPtr;`
  - `x = 12;`
- %p format specifier can be used in printf statements to print memory addresses/ pointer values
- \* and & are inverses
  - They cancel each other out



# Common Programming Error

- Dereferencing a pointer that has not been properly initialized or that has not been assigned to point to a specific location in memory is an error. This could cause a fatal execution-time error, or it could accidentally modify important data and allow the program to run to completion with incorrect results.

# A Sample Program using Pointers

```
1 // fig07_01.c
2 // Using the & and * pointer operators.
3 #include <stdio.h>
4
5 int main(void) {
6     int a = 7;
7     int *aPtr = &a; // set aPtr to the address of a
8
9     printf("Address of a is %p\nValue of aPtr is %p\n\n", &a, aPtr);
10    printf("Value of a is %d\nValue of *aPtr is %d\n\n", a, *aPtr);
11    printf("Showing that * and & are complements of each other\n");
12    printf("&*aPtr = %p\n*&aPtr = %p\n", &*aPtr, *&aPtr);
13 }
```

If **aPtr** points to **a**, then **&a** and **aPtr** have the same value.

**a** and **\*aPtr** have the same value

**&\*aPtr** and **\*&aPtr** have the same value

```
Address of a is 0x7fffe69386cc
Value of aPtr is 0x7fffe69386cc

Value of a is 7
Value of *aPtr is 7

Showing that * and & are complements of each other
&*aPtr = 0x7fffe69386cc
*&aPtr = 0x7fffe69386cc
```

**Fig. 7.1** | Using the & and \* pointer operators. (Part 2 of 2.)

# Operator Precedence

| Operators                       | Grouping      | Type           |
|---------------------------------|---------------|----------------|
| () [] ++ (postfix) -- (postfix) | left to right | postfix        |
| + - ++ -- ! * & (type)          | right to left | unary          |
| * / %                           | left to right | multiplicative |
| + -                             | left to right | additive       |
| < <= > >=                       | left to right | relational     |
| == !=                           | left to right | equality       |
| &&                              | left to right | logical AND    |
|                                 | left to right | logical OR     |
| ?:                              | right to left | conditional    |
| = += -= *= /= %=                | right to left | assignment     |
| ,                               | left to right | comma          |

# Enabling Calling Functions by Reference

- Arguments are passed to a function in two ways:
  1. pass-by-value or
  2. pass-by-reference
- Generally, arguments are passed by value
- To accomplish Pass-by-reference, address of a variable is passed as argument using & operator
- Arrays are passed by reference. Arrays are not passed with & because the array name is already a pointer

# Enabling Calling Functions by Reference

- A function that receives the address of a variable in the caller can use the indirection operator (\*) to modify the value at that location in the caller's memory, thus effecting pass-by-reference
- All changes made to the memory location are seen in the in the caller
- No need to return those values by the called function
- It enables a function to “return” multiple values by modifying caller's variable
- \* operator
  - Used as alias/nickname for variable inside of function
    - `void double( int *number )`
    - `{`
    - `*number = 2 + ( *number );`
    - `}`
  - `*number` used as nickname for the variable passed

# Pass-by-Reference with a Pointer Argument

```
1 // fig07_03.c
2 // Cube a variable using pass-by-reference with a pointer argument.
3
4 #include <stdio.h>
5
6 void cubeByReference(int *nPtr); // function prototype
7
8 int main(void) {
9     int number = 5; // initialize number
10
11     printf("The original value of number is %d", number);
12     cubeByReference(&number); // pass address of number to cubeByReference
13     printf("\nThe new value of number is %d\n", number);
14 }
15
16 // calculate cube of *nPtr; actually modifies number in main
17 void cubeByReference(int *nPtr) {
18     *nPtr = *nPtr * *nPtr * *nPtr; // cube *nPtr
19 }
```

Function prototype takes a pointer argument

Function **cubeByReference** is passed an address, which can be the value of a pointer variable

In this program, **\*nPtr** is **number**, so this statement modifies the value of **number** itself.

The original value of number is 5  
The new value of number is 125

**Fig. 7.3** | Cube a variable using pass-by-reference with a pointer argument.

# Pointer Parameters in Function Prototypes

- Pointer parameters are mentioned as pointer variables in function prototype/signature
  - `void increment(int *arg)`
  - `void newRandNum (int *num)`
- The compiler does not differentiate between a function that receives a pointer and one that receives a one-dimensional array
- If the function parameter is `int b[]`, the compiler converts the parameter to the pointer notation `int *b`.
- Similarly, if the parameter is `const int b[]`, the compiler converts the parameter to `const int *b`

# Call-by-value Analysis (part-1)

Step 1: Before main calls cubeByValue:

```
int main(void) {  
    int number = 5;  
    number = cubeByValue(number);  
}
```

number  
5

Step 2: After cubeByValue receives the call:

```
int main(void) {  
    int number = 5;  
    number = cubeByValue(number);  
}
```

number  
5

```
int cubeByValue(int n) {  
    return n * n * n;  
}
```

n  
5

Step 3: After cubeByValue cubes parameter n and before cubeByValue returns to main:

```
int main(void) {  
    int number = 5;  
    number = cubeByValue(number);  
}
```

number  
5

```
int cubeByValue(int n) {  
    return 125;  
}
```

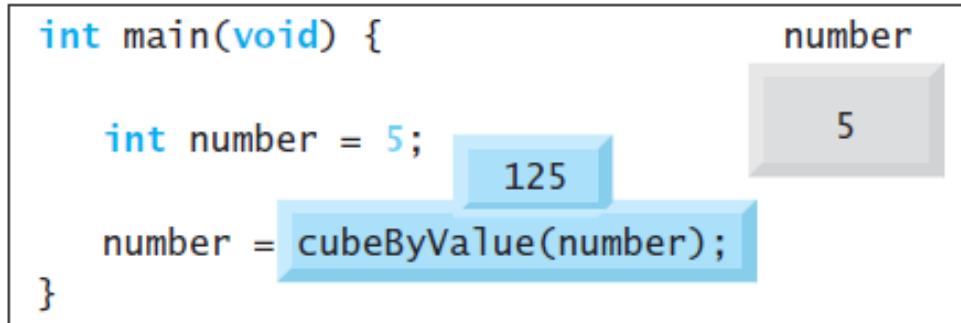
n  
5

**Fig. 7.4** | Analysis of a typical pass-by-value. (Part I of 2.)

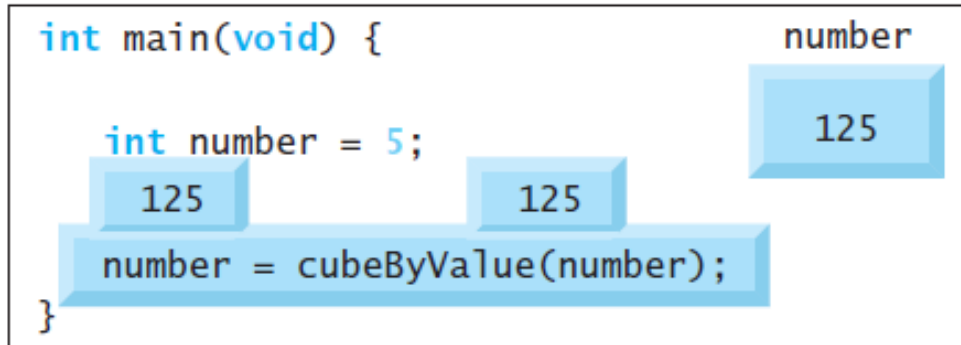


## Call-by-value Analysis (part-2)

Step 4: After `cubeByValue` returns to `main` and before assigning the result to `number`:



Step 5: After `main` completes the assignment to `number`:



**Fig. 7.4** | Analysis of a typical pass-by-value. (Part 2 of 2.)

# Call-by-reference with a pointer arg Analysis

Step 1: Before main calls cubeByReference:

```
int main(void)
{
    int number = 5;
    cubeByReference(&number);
}
```

number  
5

Step 2: After cubeByReference receives the call and before \*nPtr is cubed:

```
int main(void)
{
    int number = 5;
    cubeByReference(&number);
}
```

number  
5

```
void cubeByReference(int *nPtr)
{
    *nPtr = *nPtr * *nPtr * *nPtr;
}
```

nPtr

call establishes this pointer

Step 3: After \*nPtr is cubed and before program control returns to main:

```
int main(void)
{
    int number = 5;
    cubeByReference(&number);
}
```

number  
125

```
void cubeByReference(int *nPtr)
{
    125
    *nPtr = *nPtr * *nPtr * *nPtr;
}
```

nPtr

called function modifies caller's variable

**Fig. 7.5** | Analysis of a typical pass-by-reference with a pointer argument.

# Using call-by-reference: Guidelines

Use call-by-value to pass arguments to a function unless the caller explicitly requires the called function to modify the value of the argument variable in the caller's environment. This prevents accidental modification of the caller's arguments and is another example of the principle of least privilege.

# const Qualifier: Software Engineering Observation

- The `const` qualifier can be used to enforce the principle of least privilege. Using the principle of least privilege to properly design software reduces debugging time and improper side effects, making a program easier to modify and maintain.
- Only one value can be altered in a calling function when call-by-value is used. That value must be assigned from the return value of the function. To modify multiple values in a calling function, call-by-reference must be used.

# Using the const Qualifier with Pointers

- Recall: const qualifier
  - Variable cannot be changed
  - Use const if function does not need to change a variable (to prevent unintentional side effects)
  - Attempting to change a const variable produces an error
- const pointers
  - Point to a constant memory location
  - Must be initialized when defined
  - `int *const myPtr = &x;`
    - Type `int *const` – constant pointer to an int
  - `const int *myPtr = &x;`
    - Regular (modifiable) pointer to a const int
  - `const int *const Ptr = &x;`
    - const pointer to a const int

# Non-constant Pointer to a Non-constant Data

```
1 // fig07_06.c
2 // Converting a string to uppercase using a
3 // non-constant pointer to non-constant data.
4 #include <ctype.h>
5 #include <stdio.h>
6
7 void convertToUppercase(char *sPtr); // prototype
8
9 int main(void) {
10     char string[] = "cHaRaCters and $32.98"; // initialize char array
11
12     printf("The string before conversion is: %s\n", string);
13     convertToUppercase(string);
14     printf("The string after conversion is: %s\n", string);
15 }
16
17 // convert string to uppercase letters
18 void convertToUppercase(char *sPtr) {
19     while (*sPtr != '\0') { // current character is not null
20         *sPtr = toupper(*sPtr); // convert to uppercase
21         ++sPtr; // make sPtr point to the next character
22     }
23 }
```

Both **sPtr** and **\*sPtr** are modifiable

Both **sPtr** and **\*sPtr** are modified by the **convertToUppercase** function

```
The string before conversion is: cHaRaCters and $32.98
The string after conversion is: CHARACTERS AND $32.98
```

**Fig. 7.6** | Converting a string to uppercase using a non-constant pointer to non-constant data.

# Non-constant Pointer to a Constant Data

```
1 // fig07_07.c
2 // Printing a string one character at a time using
3 // a non-constant pointer to constant data.
4
5 #include <stdio.h>
6
7 void printCharacters(const char *sPtr);
8
9 int main(void) {
10     // initialize char array
11     char string[] = "print characters of a string";
12
13     puts("The string is:");
14     printCharacters(string);
15     puts("");
16 }
17
18 // sPtr cannot be used to modify the character to which it points,
19 // i.e., sPtr is a "read-only" pointer
20 void printCharacters(const char *sPtr) {
21     // loop through entire string
22     for (; *sPtr != '\0'; ++sPtr) { // no initialization
23         printf("%c", *sPtr);
24     }
25 }
```

Pointer variable **sPtr** is modifiable, but the data to which it points, **\*sPtr**, is not

**sPtr** is modified by function **printCharacters**

The string is:  
print characters of a string

**Fig. 7.7** | Printing a string one character at a time using a non-constant pointer to constant data.

# Non-constant inter to a Constant Data: Error

```
1 // fig07_08.c
2 // Attempting to modify data through a
3 // non-constant pointer to constant data.
4 #include <stdio.h>
5 void f(const int *xPtr); // prototype
6
7 int main(void) {
8     int y = 7; // define y
9
10    f(&y); // f attempts illegal modification
11 }
12
13 // xPtr cannot be used to modify the
14 // value of the variable to which it points
15 void f(const int *xPtr) {
16     *xPtr = 100; // error: cannot modify a const object
17 }
```

Pointer variable **xPtr** is modifiable, but the data to which it points, **\*xPtr**, is not

**\*xPtr** has the **const** qualifier, so attempting to modify its value causes an error

*Microsoft Visual C++ Error Message*

fig07\_08.c(16,5): error C2166: l-value specifies const object

**Fig. 7.8** | Attempting to modify data through a non-constant pointer to constant data.



# Constant Pointer to a Non-Constant Data

```
1 // fig07_09.c
2 // Attempting to modify a constant pointer to non-constant data.
3 #include <stdio.h>
4
5 int main(void) {
6     int x = 0; // define x
7     int y = 0; // define y
8
9     // ptr is a constant pointer to an integer that can be modified
10    // through ptr, but ptr always points to the same memory location
11    int * const ptr = &x;
12
13    *ptr = 7; // allowed: *ptr is not const
14    ptr = &y; // error: ptr is const; cannot assign new address
15 }
```

Pointer **ptr** is not modifiable, but the data to which it points, **\*ptr**, can be changed

*Microsoft Visual C++ Error Message*

fig07\_09.c(14,4): error C2166: l-value specifies const object

**Fig. 7.9** | Attempting to modify a constant pointer to non-constant data.

# Constant Pointer to a Constant Data: Error

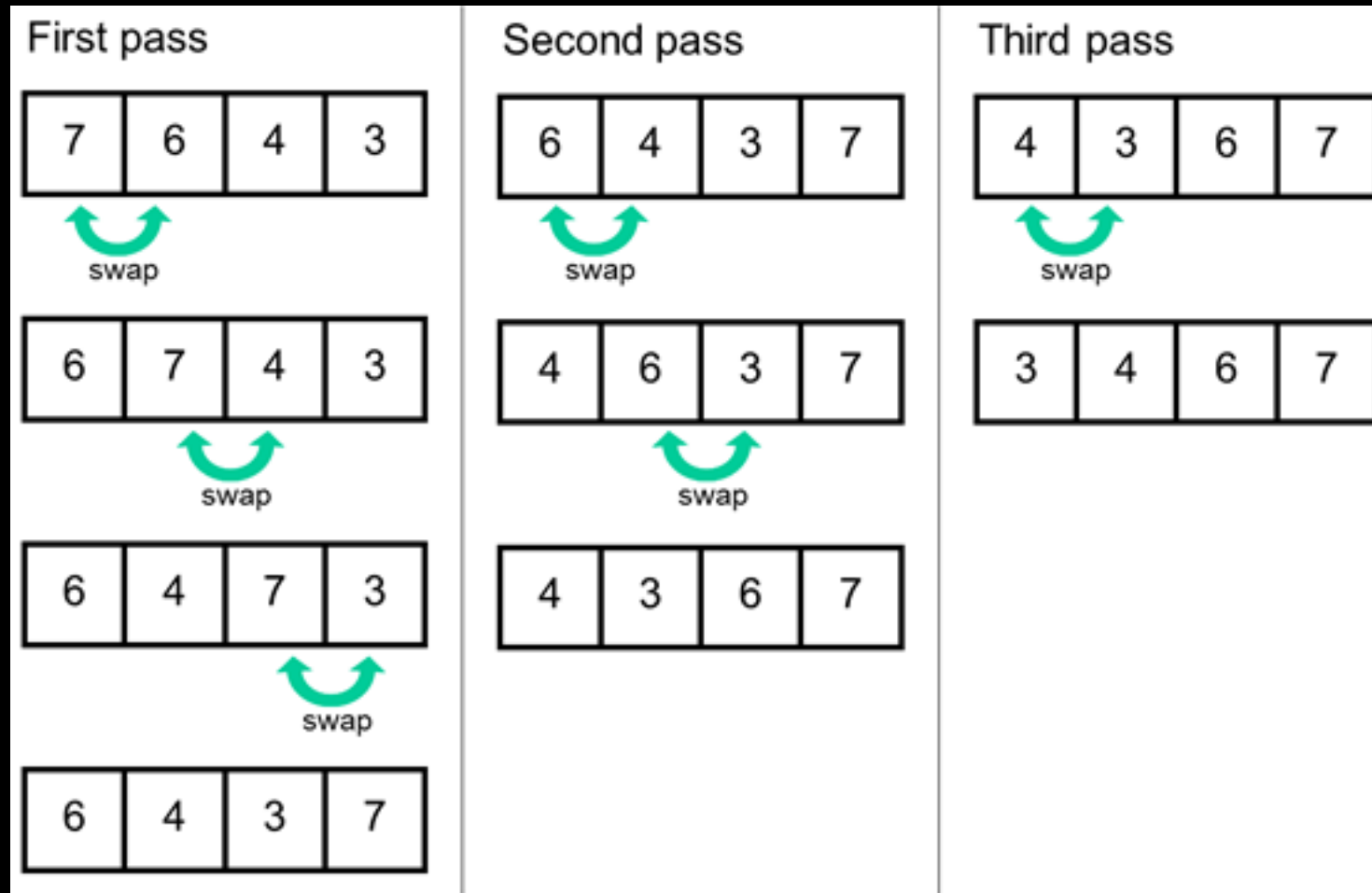
```
1 // fig07_10.c
2 // Attempting to modify a constant pointer to constant data.
3 #include <stdio.h>
4
5 int main(void) {
6     int x = 5;
7     int y = 0;
8
9     // ptr is a constant pointer to a constant integer. ptr always
10    // points to the same location; the integer at that location
11    // cannot be modified
12    const int *const ptr = &x; // initialization is OK
13
14    printf("%d\n", *ptr);
15    *ptr = 7; // error: *ptr is const; cannot assign new value
16    ptr = &y; // error: ptr is const; cannot assign new address
17 }
```

Neither pointer **sPtr** nor the data to which it points, **\*sPtr**, is modifiable

*Microsoft Visual C++ Error Message*

```
fig07_10.c(15,5): error C2166: l-value specifies const object
fig07_10.c(16,4): error C2166: l-value specifies const object
```

# Bubble Sort Using Call-by-Reference



# Bubble Sort Using Call-by-Reference

- Implement bubble sort using pointers
  - Swap two elements
  - swap function must receive address (using &) of array elements
  - Using pointers and the \* operator, swap can switch array elements

- Psuedocode

*Initialize array*

*print data in original order*

*Call function bubble sort*

*print sorted array*

*Define bubble sort*

# Bubble Sort Using Call-by-Reference

```
1 // fig07_11.c
2 // Putting values into an array, sorting the values into
3 // ascending order and printing the resulting array.
4 #include <stdio.h>
5 #define SIZE 10
6
7 void bubbleSort(int * const array, size_t size); // prototype
8
9 int main(void) {
10     // initialize array a
11     int a[SIZE] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
12
13     puts("Data items in original order");
14
15     // loop through array a
16     for (size_t i = 0; i < SIZE; ++i) {
17         printf("%4d", a[i]);
18     }
19
20     bubbleSort(a, SIZE); // sort the array
21
22     puts("\nData items in ascending order");
23
24     // loop through array a
25     for (size_t i = 0; i < SIZE; ++i) {
26         printf("%4d", a[i]);
27     }
28
29     puts("");
30 }
31
```

# Bubble Sort Using Call-by-Reference

```
28
29     puts("");
30 }
31
32 // sort an array of integers using bubble sort algorithm
33 void bubbleSort(int * const array, size_t size) {
34     void swap(int *element1Ptr, int *element2Ptr); // prototype
35
36     // loop to control passes
37     for (int pass = 0; pass < size - 1; ++pass) {
38         // loop to control comparisons during each pass
39         for (size_t j = 0; j < size - 1; ++j) {
40             // swap adjacent elements if they're out of order
41             if (array[j] > array[j + 1]) {
42                 swap(&array[j], &array[j + 1]);
43             }
44         }
45     }
46 }
47
```

**Fig. 7.11** | Putting values into an array, sorting the values into ascending order and printing

# Software Engineering Observation

- When passing an array to a function, also pass the size of the array. This helps make the function reusable in many programs.
- Global variables usually violate the principle of least privilege and can lead to poor software engineering. Global variables should be used only to represent truly shared resources, such as the time of day.

# Bubble Sort Using Call-by-Reference

```
48 // swap values at memory locations to which element1Ptr and
49 // element2Ptr point
50 void swap(int *element1Ptr, int *element2Ptr) {
51     int hold = *element1Ptr;
52     *element1Ptr = *element2Ptr;
53     *element2Ptr = hold;
54 }
```

Function **swap** changes the values of the **ints** that the two pointers point to

Data items in original order  
2 6 4 8 10 12 89 68 45 37  
Data items in ascending order  
2 4 6 8 10 12 37 45 68 89

**Fig. 7.11** | Putting values into an array, sorting the values into ascending order and printing the resulting array. (Part 2 of 2.)



# sizeof Operator

- `sizeof`
  - Returns size of operand in bytes
  - For arrays: size of one element \* number of elements
  - if `sizeof( int )` equals 4 bytes, then
    - `int myArray[ 10 ];`
    - `printf( "%d", sizeof( myArray ) );`
    - will print 40 – array's total number of bytes
- `sizeof` can be used with
  - Variable names
  - Type name
  - Constant values

# sizeof Operator: Example

```
1  /* Fig. 7.16: fig07_16.c
2     sizeof operator when used on an array name
3     returns the number of bytes in the array. */
4  #include <stdio.h>
5
6  size_t getSize( float *ptr ); /* prototype */
7
8  int main( void )
9  {
10     float array[ 20 ]; /* create array */
11
12     printf( "The number of bytes in the array is %d"
13            "\nThe number of bytes returned by getSize is %d\n",
14            sizeof( array ), getSize( array ) );
15
16     return 0; /* indicates successful termination */
17
18 } /* end main */
19
20 /* return size of ptr */
21 size_t getSize( float *ptr )
22 {
23     return sizeof( ptr );
24
25 } /* end function getSize */
```

← floats take up 4 bytes in memory, so 20 floats take up 80 bytes

```
The number of bytes in the array is 80
The number of bytes returned by getSize is 4
```

# sizeof Operator: Performance and Portability

- `sizeof` is a compile-time operator, so it does not incur any execution-time overhead.
- The number of bytes used to store a particular data type may vary between systems. When writing programs that depend on data type sizes and that will run on several computer systems, use `sizeof` to determine the number of bytes used to store the data types.

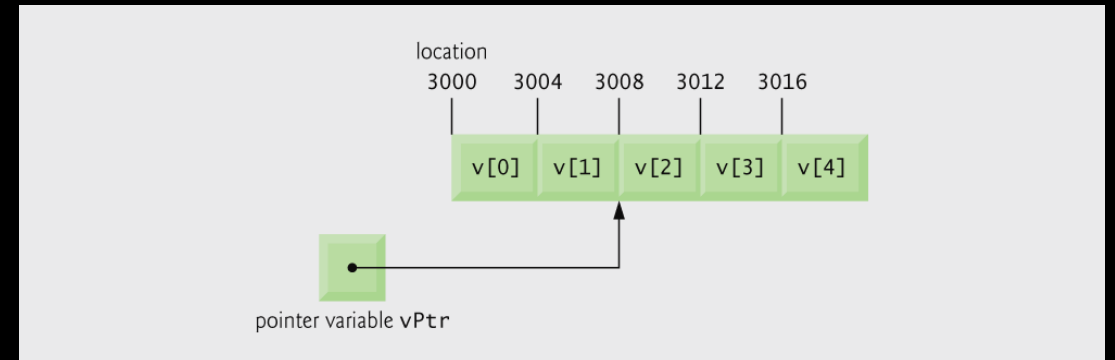
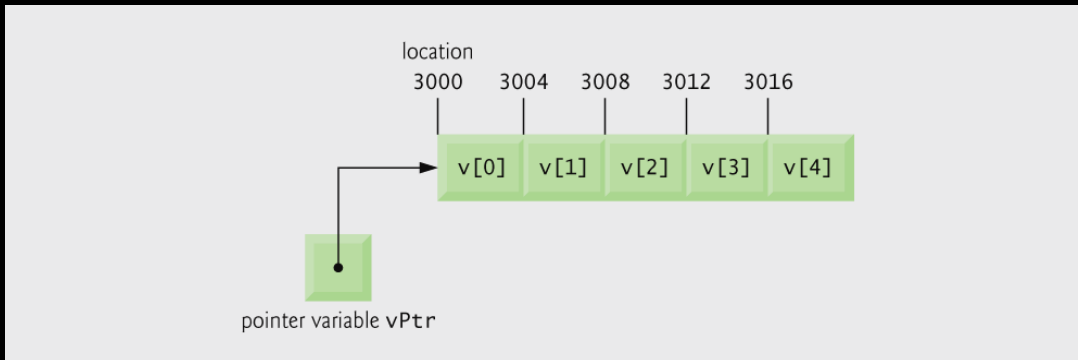
# Pointer Expressions and Pointer Arithmetic

- Arithmetic operations can be performed on pointers
  - Increment/decrement pointer (++ or --)
  - Add an integer to a pointer( + or += , - or -=)
  - Pointers may be subtracted from each other
  - Operations meaningless unless performed on an array



# Pointer Expressions and Pointer Arithmetic

- 5 element `int` array on machine with 4 byte `ints`
  - `vPtr` points to first element `v[ 0 ]`
    - at location 3000 (`vPtr = 3000`)
  - `vPtr += 2`; sets `vPtr` to 3008
    - `vPtr` points to `v[ 2 ]` (incremented by 2), but the machine has 4 byte `ints`, so it points to address 3008



# Pointer Expressions and Pointer Arithmetic: Portability

- Most computers today have 2-byte or 4-byte integers. Some of the newer machines use 8-byte integers. Because the results of pointer arithmetic depend on the size of the objects a pointer points to, pointer arithmetic is machine dependent.

# Pointer Expressions and Pointer Arithmetic: Errors

- Using pointer arithmetic on a pointer that does not refer to an element in an array.
- Subtracting or comparing two pointers that do not refer to elements in the same array.
- Running off either end of an array when using pointer arithmetic.



# Pointer Expressions and Pointer Arithmetic

- Pointers of the same type can be assigned to each other
  - If not the same
  - `+type`, a cast operator must be used
  - Exception: pointer to `void` (`type void *`)
    - Generic pointer, represents any type
    - No casting needed to convert a pointer to `void` pointer
    - `void` pointers cannot be dereferenced
- Assigning a pointer of one type to a pointer of another type if neither is of type `void *` is a syntax error.
- Dereferencing a `void *` pointer is a syntax error.

# Pointers and Arrays: Relationship

- Arrays and pointers closely related
  - Array name as a constant pointer to the array's first element
  - Pointers can do array subscripting operations
- Define an array `b[ 5 ]` and a pointer `bPtr`
  - To set them equal to one another use:
    - `bPtr = b;`
    - The array name (`b`) is actually the address of first element of the array `b[ 5 ]`
      - `bPtr = &b[ 0 ]`
    - Explicitly assigns `bPtr` to address of first element of `b`

# Pointers and Arrays: Pointer/Offset Notation

- Element `b[ 3 ]`
  - Can be accessed by `*( bPtr + 3 )`
    - Where 3 is called the offset. Called pointer/offset notation
  - Can be accessed by performing pointer arithmetic on the array itself
    - `*( b + 3 )`

# Pointers and Arrays: Pointer/Subscript Notation

- Element `b[ 3 ]`
  - Can be accessed by `bptr[ 3 ]`
    - Called pointer/subscript notation
    - `bPtr[ 3 ]` same as `b[ 3 ]`
- Attempting to modify an array name with pointer arithmetic is a syntax error.
  - `b += 3`

# Pointers and Arrays: Example-1

```
1  /* Fig. 7.20: fig07_20.cpp
2     Using subscripting and pointer notations with arrays */
3
4  #include <stdio.h>
5
6  int main( void )
7  {
8      int b[] = { 10, 20, 30, 40 }; /* initialize array b */
9      int *bPtr = b;                /* set bPtr to point to array b */
10     int i;                        /* counter */
11     int offset;                   /* counter */
12
13     /* output array b using array subscript notation */
14     printf( "Array b printed with:\nArray subscript notation\n" );
15
16     /* loop through array b */
17     for ( i = 0; i < 4; i++ ) {
18         printf( "b[ %d ] = %d\n", i, b[ i ] );
19     } /* end for */
20
21     /* output array b using array name and pointer/offset notation */
22     printf( "\nPointer/offset notation where\n"
23             "the pointer is the array name\n" );
24
25     /* loop through array b */
26     for ( offset = 0; offset < 4; offset++ ) {
27         printf( "( b + %d ) = %d\n", offset, *( b + offset ) );
28     } /* end for */
29
```

Array subscript notation

Pointer/offset notation

# Pointers and Arrays: Example-1

```
30  /* output array b using bPtr and array subscript notation */
31  printf( "\nPointer subscript notation\n" );
32
33  /* loop through array b */
34  for ( i = 0; i < 4; i++ ) {
35      printf( "bPtr[ %d ] = %d\n", i, bPtr[ i ] );
36  } /* end for */
37
38  /* output array b using bPtr and pointer/offset notation */
39  printf( "\nPointer/offset notation\n" );
40
41  /* loop through array b */
42  for ( offset = 0; offset < 4; offset++ ) {
43      printf( "*( bPtr + %d ) = %d\n", offset, *( bPtr + offset ) );
44  } /* end for */
45
46  return 0; /* indicates successful termination */
47
48 } /* end main */
```

Pointer subscript notation

Pointer offset notation

Array b printed with:  
Array subscript notation  
b[ 0 ] = 10  
b[ 1 ] = 20  
b[ 2 ] = 30  
b[ 3 ] = 40

(continued on next slide...)

# Pointers and Arrays: Example-1

*(continued from previous slide...)*

Pointer/offset notation where  
the pointer is the array name

`*( b + 0 ) = 10`

`*( b + 1 ) = 20`

`*( b + 2 ) = 30`

`*( b + 3 ) = 40`

Pointer subscript notation

`bPtr[ 0 ] = 10`

`bPtr[ 1 ] = 20`

`bPtr[ 2 ] = 30`

`bPtr[ 3 ] = 40`

Pointer/offset notation

`*( bPtr + 0 ) = 10`

`*( bPtr + 1 ) = 20`

`*( bPtr + 2 ) = 30`

`*( bPtr + 3 ) = 40`

# Pointers and Arrays: Example-2

```
1  /* Fig. 7.21: fig07_21.c
2     Copying a string using array notation and pointer notation. */
3  #include <stdio.h>
4
5  void copy1( char * const s1, const char * const s2 ); /* prototype */
6  void copy2( char *s1, const char *s2 ); /* prototype */
7
8  int main( void )
9  {
10     char string1[ 10 ];          /* create array string1 */
11     char *string2 = "Hello";     /* create a pointer to a string */
12     char string3[ 10 ];          /* create array string3 */
13     char string4[] = "Good Bye"; /* create a pointer to a string */
14
15     copy1( string1, string2 );
16     printf( "string1 = %s\n", string1 );
17
18     copy2( string3, string4 );
19     printf( "string3 = %s\n", string3 );
20
21     return 0; /* indicates successful termination */
22
23 } /* end main */
24
```



## Pointers and Arrays: Example-2

```
25 /* copy s2 to s1 using array notation */
26 void copy1( char * const s1, const char * const s2 )
27 {
28     int i; /* counter */
29
30     /* loop through strings */
31     for ( i = 0; ( s1[ i ] = s2[ i ] ) != '\0'; i++ ) {
32         ; /* do nothing in body */
33     } /* end for */
34
35 } /* end function copy1 */
36
37 /* copy s2 to s1 using pointer notation */
38 void copy2( char *s1, const char *s2 )
39 {
40     /* loop through strings */
41     for ( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ ) {
42         ; /* do nothing in body */
43     } /* end for */
44
45 } /* end function copy2 */
```

Condition of **for** loop  
actually performs an action

```
string1 = Hello
string3 = Good Bye
```

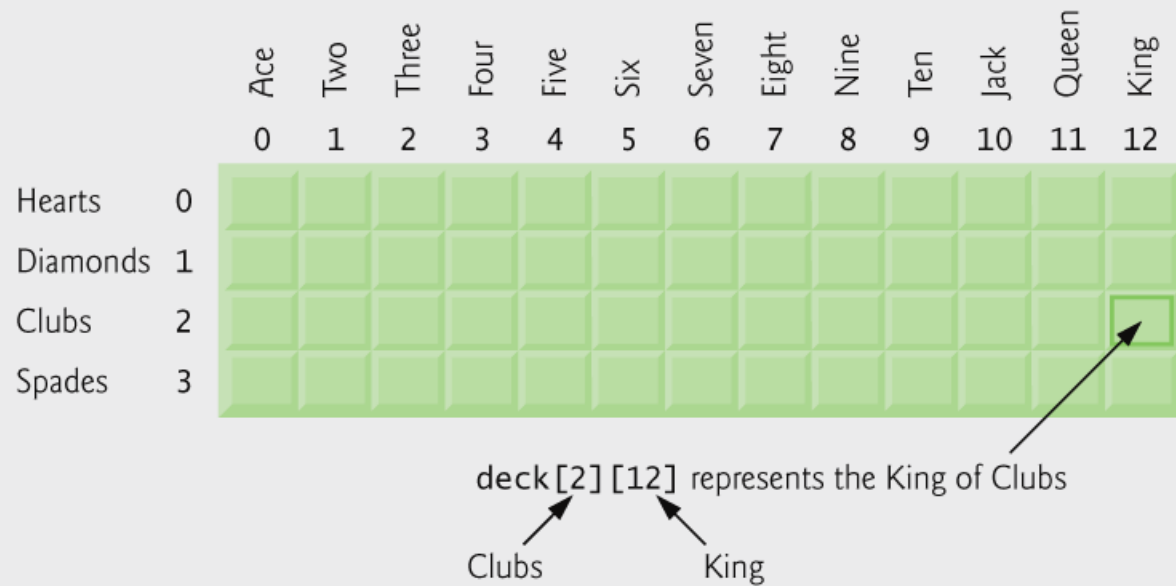
# Arrays of Pointers

- Arrays can contain pointers
- For example: an array of strings

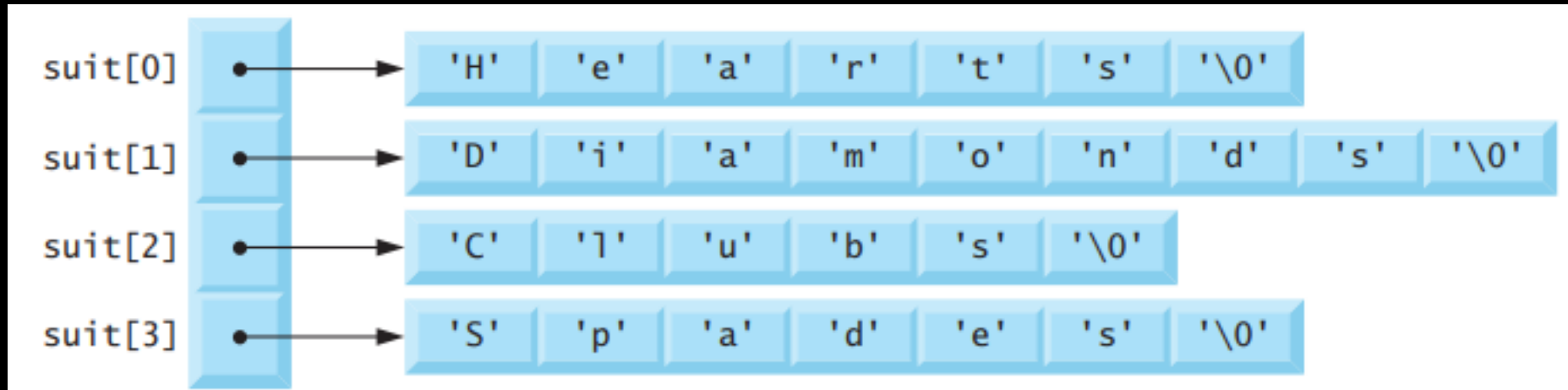
```
char *suit[ 4 ] = { "Hearts", "Diamonds", "Clubs", "Spades" };
```

- Strings are pointers to the first character
- `char *` – each element of `suit` is a pointer to a char
- The strings are not actually stored in the array `suit`, only pointers to the strings are stored

# Arrays of Pointers: Another Example: Graphical Representation



# Arrays of Pointers: Graphical Representation



# Pointers to Functions (Function Pointers)

- Pointer to function
  - Contains address of function
  - Similar to how array name is address of first element
  - Function name is starting address of code that defines function
- Function pointers can be
  - Passed to functions
  - Stored in arrays
  - Assigned to other function pointers

# Function Pointers: Example-1

```
1  /* Fig. 7.26: fig07_26.c
2     Multipurpose sorting program using function pointers */
3  #include <stdio.h>
4  #define SIZE 10
5
6  /* prototypes */
7  void bubble( int work[], const int size, int (*compare)( int a, int b ) );
8  int ascending( int a, int b );
9  int descending( int a, int b );
10
11 int main( void )
12 {
13     int order;    /* 1 for ascending order or 2 for descending order */
14     int counter; /* counter */
15
16     /* initialize array a */
17     int a[ SIZE ] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
18
19     printf( "Enter 1 to sort in ascending order,\n"
20            "Enter 2 to sort in descending order: " );
21     scanf( "%d", &order );
22
23     printf( "\nData items in original order\n" );
24
25     /* output original array */
26     for ( counter = 0; counter < SIZE; counter++ ) {
27         printf( "%5d", a[ counter ] );
28     } /* end for */
29
```

**bubble** function takes a function pointer as an argument

# Function Pointers: Example-1

```
30  /* sort array in ascending order; pass function ascending as an
31     argument to specify ascending sorting order */
32  if ( order == 1 ) {
33      bubble( a, SIZE, ascending );
34      printf( "\nData items in ascending order\n" );
35  } /* end if */
36  else { /* pass function descending */
37      bubble( a, SIZE, descending );
38      printf( "\nData items in descending order\n" );
39  } /* end else */
40
41  /* output sorted array */
42  for ( counter = 0; counter < SIZE; counter++ )
43      printf( "%5d", a[ counter ] );
44  } /* end for */
45
46  printf( "\n" );
47
48  return 0; /* indicates successful termination */
49
50 } /* end main */
51
```

depending on the user's choice, the **bubble** function uses either the **ascending** or **descending** function to sort the array

# Function Pointers: Example-1

```
52 /* multipurpose bubble sort; parameter compare is a pointer to
53    the comparison function that determines sorting order */
54 void bubble( int work[], const int size, int (*compare)( int a, int b ) )
55 {
56     int pass; /* pass counter */
57     int count; /* comparison counter */
58
59     void swap( int *element1Ptr, int *element2ptr ); /* prototype */
60
61     /* loop to control passes */
62     for ( pass = 1; pass < size; pass++ ) {
63
64         /* loop to control number of comparisons per pass */
65         for ( count = 0; count < size - 1; count++ ) {
66
67             /* if adjacent elements are out of order, swap them */
68             if ( (*compare)( work[ count ], work[ count + 1 ] ) ) {
69                 swap( &work[ count ], &work[ count + 1 ] );
70             } /* end if */
71
72         } /* end for */
73
74     } /* end for */
75
76 } /* end function bubble */
77
```

Note that what the program considers “out of order” is dependent on the function pointer that was passed to the **bubble** function



# Function Pointers: Example-1

```
78 /* swap values at memory locations to which element1Ptr and
79    element2Ptr point */
80 void swap( int *element1Ptr, int *element2Ptr )
81 {
82     int hold; /* temporary holding variable */
83
84     hold = *element1Ptr;
85     *element1Ptr = *element2Ptr;
86     *element2Ptr = hold;
87 } /* end function swap */
```

```
88
89 /* determine whether elements are out of order for an ascending
90    order sort */
```

```
91 int ascending( int a, int b )
92 {
93     return b < a; /* swap if b is less than a */
94
95 } /* end function ascending */
```

Passing the **bubble** function **ascending**  
will point the program here

```
96
97 /* determine whether elements are out of order for a descending
98    order sort */
```

```
99 int descending( int a, int b )
100 {
101     return b > a; /* swap if b is greater than a */
102
103 } /* end function descending */
```

Passing the **bubble** function **descending**  
will point the program here

# Function Pointers: Example-1

Enter 1 to sort in ascending order,  
Enter 2 to sort in descending order: 1

Data items in original order

2    6    4    8    10    12    89    68    45    37

Data items in ascending order

2    4    6    8    10    12    37    45    68    89

Enter 1 to sort in ascending order,  
Enter 2 to sort in descending order: 2

Data items in original order

2    6    4    8    10    12    89    68    45    37

Data items in descending order

89    68    45    37    12    10    8    6    4    2

# Function Pointers: Example-2

```
1  /* Fig. 7.28: fig07_28.c
2     Demonstrating an array of pointers to functions */
3  #include <stdio.h>
4
5  /* prototypes */
6  void function1( int a );
7  void function2( int b );
8  void function3( int c );
9
10 int main( void )
11 {
12     /* initialize array of 3 pointers to functions that each take an
13        int argument and return void */
14     void (*f[ 3 ])( int ) = { function1, function2, function3 };
15
16     int choice; /* variable to hold user's choice */
17
18     printf( "Enter a number between 0 and 2, 3 to end: " );
19     scanf( "%d", &choice );
20
```

Array of pointers to functions

# Function Pointers: Example-2

```
21  /* process user's choice */
22  while ( choice >= 0 && choice < 3 ) {
23
24      /* invoke function at location choice in array f and pass
25         choice as an argument */
26      (*f[ choice ])( choice );
27
28      printf( "Enter a number between 0 and 2, 3 to end: " );
29      scanf( "%d", &choice );
30  } /* end while */
31
32  printf( "Program execution completed.\n" );
33
34  return 0; /* indicates successful termination */
35
36 } /* end main */
37
38 void function1( int a )
39 {
40     printf( "You entered %d so function1 was called\n\n", a );
41 } /* end function1 */
42
43 void function2( int b )
44 {
45     printf( "You entered %d so function2 was called\n\n", b );
46 } /* end function2 */
```

Function called is dependent on user input

# Function Pointers: Example-2

```
47
48 void function3( int c )
49 {
50     printf( "You entered %d so function3 was called\n\n", c );
51 } /* end function3 */
```

Enter a number between 0 and 2, 3 to end: 0  
You entered 0 so function1 was called

Enter a number between 0 and 2, 3 to end: 1  
You entered 1 so function2 was called

Enter a number between 0 and 2, 3 to end: 2  
You entered 2 so function3 was called

Enter a number between 0 and 2, 3 to end: 3  
Program execution completed.

# Dynamic Memory Management

- Creating and maintaining dynamic data structures that grow and shrink at execution time
  - Allocate memory at execution time (malloc)
  - Release memory no longer needed (free)
- malloc takes as parameter the number of bytes to allocate
- Returns a void pointer, NULL is allocation fails
  - `myVarPtr=(int *) malloc (sizeof(int));`
  - `myArrPtr=(int *) malloc (sizeof(int)*10);`
- The function free can be used to release the allocated memory. Takes the memory address (pointer) as an argument.
  - `free(myArrPtr);`

# Dynamic Memory Mgmt: Example

```
1 // Pointes and 2D Arrays
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main() {
6     int *a;
7     char *arr;
8     a = (int *) malloc (sizeof(int));
9     arr = (char *) malloc (sizeof(char)*5);
10
11     *a=2;
12     *arr='P'; *(arr+1)='R';
13     arr[2]='0'; arr[3]='G'; *(&arr[3]+1)='\0';
14
15     printf("a=%p, arr=%p\n", a, arr);
16     printf("a=%d arr=", *a);
17     puts(arr);
18     free(a); free(arr);
19 }
20
```

```
a=0x1f452a0, arr=0x1f452c0
a=2 arr=PROG
```

# Dynamic Memory Mgmt: Example

- Reallocate memory

```
ptr = realloc (ptr,newsiz
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {  
    int *ptr;  
    ptr = malloc(200);  
    if (ptr == NULL) {  
        printf("The memory is not allocated.");  
        exit(0);  
    }  
    ptr = realloc(ptr,400);  
    if(ptr!= NULL)  
        printf("Memory has successfully allocated\n");  
    return 0;  
}
```